Развитие META до версии 4.0

Подробный отчёт о работе в этом чате и исправлении 4.0.1

Для владельца локальной META • 2 ноября 2026 года

Главный результат

В этом чате мы создали устанавливаемое обновление существующей Windows-системы META Core и Desktop. META получила отдельный агентный режим, в котором локальная модель выбирает инструменты, изучает проект, готовит изменения и при разрешении запускает Python-тесты. Результат работы передаётся человеку общей файловой заявкой с просмотром различий.

Это расширение существующей META, а не новая модель искусственного интеллекта. Вычисления по-прежнему выполняет Ollama, в том числе установленная qwen2.5-coder:14b. Прирост возможностей получен за счёт программного окружения модели: инструментов, обратной связи от тестов, ограничений исполнения, состояния задач и связи с интерфейсом.

Что удалось подтвердить

Пройдены 17 автоматических проверок. Отдельно проведён настоящий запуск на локальной Qwen 14B: модель прочитала два файла, исправила ошибку сложения, успешно запустила unittest и закончила работу за пять шагов. Исходный файл примера при этом не изменился: правки остались в предлагаемом результате.

После сообщения «Project directory must already exist» мы выпустили и применили исправление 4.0.1 к установленным Core и Desktop. Оно добавило создание новой папки проекта из панели. Перед заменой четырёх файлов сохранена резервная копия. Для использования исправления необходим перезапуск Core и Desktop.

Как понимать сравнение с 3.9

В запросе предыдущая версия названа 3.9. Однако обнаруженный в начале чата движок имеет явную маркировку META 3.8, рядом присутствуют компоненты 3.6 и 3.7. Отдельная подтверждённая сборка 3.9 в этом чате не исследовалась. Поэтому далее сравнивается прежняя установленная линия 3.x с реализованной 4.0; это не утверждение о неизвестных возможностях отдельной версии 3.9.

Навигация по отчёту

Далее разобраны исходное состояние и ход работ, устройство агента, сравнение возможностей, ограничения исполнения, результаты проверок, установка и откат, исправление 4.0.1, оставшиеся задачи и состав файлов.

1 Исходное состояние и ход работы

С чего мы начали

Пользователь попросил заметное обновление за один проход: приблизить META к рабочему циклу помощника программиста, а не ограничиться планом или сменой номера версии. В задачу входили ограниченный агентный цикл, инструменты проекта, управляемый запуск тестов, Ollama, многофайловые изменения, связь с Core и Desktop, проверка установки и откат. Ориентир «около 70 процентов Claude Code» был целью по набору возможностей, а не измеренным результатом.

Мы прочитали доступный контекст предыдущего разговора и реальные исходники по путям D:\META\Core и C:\Users\user\Downloads\META_1_3_Hotfix. В них уже существовали локальный чат, работа с файлами, предложения изменений, подтверждение через Desktop, многофайловые заявки и движок статической проверки с ограниченными попытками исправления.

Что уже было до 4.0

Компонент agent_project37.py объединял несколько файлов в одну заявку и проверял их перед применением. Модуль meta_engine38.py разбирал Python и JSON, проверял размер и пути файлов и мог до двух раз обратиться к Ollama для исправления обнаруженных ошибок. Предыдущая версия уже имела полезные ограничения, проверки хэшей и элементы восстановления после ошибок записи. Эти возможности не следует приписывать исключительно 4.0.

Что мы написали заново

Для нового режима создан пакет meta40 из отдельных модулей рабочего пространства, агентного цикла, запуска тестов, заявок и API. Вместо ожидания одного большого текстового ответа модель получает возможность по очереди вызывать инструменты. Ответ каждого инструмента возвращается ей в следующий ход. Так появилась основа цикла «прочитать → изменить → проверить → исправить».

К Core подключены новые маршруты /agent40. В Desktop добавлены серверный посредник и отдельная панель «Агент 4.0». Связь с существующим agent_pending сохранена: прежние заявки продолжают обрабатываться старым механизмом, а заявки нового типа передаются новому обработчику.

Как мы довели результат до установки

Подготовлены установщик, предварительная проверка совместимости, резервные копии с контрольными суммами, восстановление при ошибке установки и отдельная команда отката. Выпущены ZIP-архив, инструкция, тестовый проект и отчёт настоящего запуска Ollama. Полная установка проверялась на временных образцах контрактов приложения; совместимость преобразований дополнительно проверялась на реальных исходниках.

2 Как устроен агентный режим

Задача становится последовательностью действий

Пользователь выбирает проект, модель, число шагов и разрешение на тесты. Core проверяет параметры, создаёт запись задачи и запускает обработку в фоновом потоке. Одновременно выполняется одна задача. Интерфейс может получать её состояние, показывать журнал и отправлять запрос на остановку.

В начале создаётся снимок разрешённых текстовых файлов выбранного проекта. Модель получает задачу, список файлов, установленный бюджет и последние сохранённые сведения по этому проекту. Само содержимое файла она запрашивает отдельным чтением. Перед изменением существующего файла требуется сначала прочитать его.

Инструменты модели

list показывает список доступных файлов; read возвращает текст выбранного файла; search ищет буквальную строку и возвращает совпадения с номерами строк. write сохраняет полное новое содержимое файла в отдельном слое правок. test запускает единственный разрешённый профиль python-unittest. finish завершает работу и пытается создать общую заявку.

Модель возвращает одно действие в JSON за ход. Неизвестные действия, неверные пути, слишком большие файлы и ошибки синтаксиса превращаются в сообщения об ошибке инструмента. Модель видит эти сообщения и может исправить следующий шаг. При исчерпании времени или числа шагов задача останавливается без автоматического применения изменений.

Почему рабочие файлы не меняются во время размышления

Снимок исходников и слой предлагаемых изменений разделены. Агент может несколько раз переписать один файл, а тестовая копия будет собрана из исходного снимка и последних правок. Поэтому неудачные промежуточные версии не записываются в рабочий проект. Исключение касается только пустых каталогов: в 4.0.1 выбранная новая папка может создаваться при запуске по разрешению пользователя.

Что происходит при завершении

Если есть реальные изменения, создаётся заявка типа meta40 в существующем хранилище agent_pending. Она содержит пути, новые тексты и ожидаемые хэши прежнего содержимого. Desktop показывает diff — разницу между текущими и предлагаемыми файлами. Пользователь подтверждает всю заявку или отклоняет её.

При создании заявки и перед записью проверяется, что исходные файлы не изменились. Это защищает от перезаписи более поздних правок пользователя. Успешное применение сохраняет квитанцию с прежним содержимым; при ошибке нескольких замен предпринимается восстановление уже записанных файлов.

Что означает память проекта

Между задачами сохраняются последняя задача и краткое резюме модели. Это небольшой перенос контекста, а не база знаний по всему репозиторию. Память считается недоверенными данными. Резюме задачи со статусом pending не доказывает, что предложенные изменения действительно применены.

3 Сравнение прежней линии и версии 4.0

Столбец «Прежняя линия 3.x» описывает изученные компоненты 3.6–3.8. Использовать его как точную спецификацию отдельной 3.9 без её проверки нельзя. Столбец 4.0 учитывает установленное исправление 4.0.1.

Область	Прежняя линия 3.x	Версия 4.0 и исправление 4.0.1
Основной сценарий	Генерация ответа и проверка предложенных файлов	Последовательный выбор действий с обратной связью от инструментов
Самостоятельность	Заранее заданная обработка и ограниченный ремонт	Модель решает, что прочитать, найти, изменить и проверить следующим
Контекст проекта	Список файлов и ограниченный контекст отдельных целей	Снимок проекта, чтение по запросу, обязательное чтение перед правкой
Поиск в коде	Не был частью нового управляемого цикла	Буквальный поиск с путём и номером строки
Проверка кода	Python AST, JSON и ограничения файлов	Сохранены синтаксические проверки; добавлен реальный unittest
Исправление ошибок	До двух попыток ремонта статических ошибок в 3.8	Реакция на ошибки инструментов и вывод тестов в пределах общего бюджета
Команды	Генерируемый код не исполнялся движком 3.8	Один профиль Python unittest по отдельному разрешению
Правки нескольких файлов	Общие заявки уже были с 3.7	Заявка формируется из накопленных правок агентного цикла
Подтверждение	Предпросмотр и применение через Desktop	Прежняя точка подтверждения плюс карточка новой панели
Восстановление	Резервирование и попытка отката уже были	Новая квитанция с прежними байтами и отдельный CLI отката заявки
Интерфейс агента	Результат преимущественно в чате	Журнал действий, состояние задачи, тесты, остановка и история
Контекст между задачами	Общая память META уже существовала	Отдельное краткое резюме последней задачи по проекту
Новый проект	Нужна заранее созданная папка	В 4.0.1 папка создаётся из панели по разрешению
Установка	Предыдущие обновления и резервные копии	Проверка контрактов, манифест файлов, контрольные суммы и откат
Git и свободный терминал	Не подтверждены этим чатом	Не реализованы и в новом режиме

Главное изменение — появление обратной связи от реального выполнения тестов внутри ограниченного цикла. Проверки путей, подтверждение человеком и многофайловые заявки развивают уже существовавшую основу.

4 Ограничения и безопасность исполнения

Управляемый запуск тестов

По умолчанию выполнение кода выключено. Пользователь должен разрешить Python unittest для конкретной задачи и подтвердить это в панели. Сервер проверяет не только флаг, но и специальное значение согласия. Модель выбирает профиль проверки, а не исполняемый файл, командную строку или произвольные аргументы.

Запуск идёт через существующий Python META без shell. Собирается временная копия разрешённых файлов проекта и текущих правок. Передаются только выбранные служебные переменные окружения. Ноль обнаруженных тестов считается неуспехом; успешный результат требует реального завершения тестового процесса с соответствующим кодом.

На Windows используется Job Object: он ограничивает число процессов и память одного процесса, а закрытие объекта завершает дерево дочерних процессов. Старт тестового кода задерживается, пока процесс не присоединён к этому объекту. Если присоединение невозможно, тест не должен продолжаться.

Чего эти меры не гарантируют

Временная копия не является защитной песочницей операционной системы. Исполняемый Python-код работает с правами пользователя и может обращаться к другим файлам и сети. Ограничение списка команд не превращает недоверенный тест в безопасный код. Поэтому разрешение исполнения предназначено для доверенных проектов.

Ограничение	Реализованное значение
Шаги агента	От 1 до 30; по умолчанию 16
Бюджет задачи	По умолчанию 600 секунд; в API от 30 до 1200
Один запуск тестов	До 30 секунд; не больше остатка бюджета цикла
Количество запусков	До четырёх за задачу
Вывод одного запуска	До 32 000 байт
Набор правок	До 12 файлов; до 20 000 байт каждый; до 100 000 вместе
Снимок проекта	До 1000 файлов и 8 000 000 байт; предел обхода 2000 каталогов
Один запрос модели	Тайм-аут до 60 секунд

Тайм-аут запроса Ollama не является строгим мгновенным прерыванием всей задачи. Отмена может ждать завершения уже начатого запроса; подготовка снимка также занимает время. Эти пределы ограничивают работу, но не обещают остановку ровно в заданную секунду.

Границы файлов и API

Запрещены выход за Projects, абсолютные пути файлов, Windows ADS, имена устройств, символические ссылки и junction. Скрытые файлы, крупные файлы, двоичные данные и некоторые имена, похожие на секреты, исключаются. Новые маршруты Core защищены токеном; Desktop хранит его на серверной стороне и проверяет вход, Host и Origin. Это относится к новому режиму, а не является полным аудитом всех старых маршрутов META.

5 Что именно проверено

Автоматические проверки

Итоговая версия прошла 17 автоматических тестовых сценариев в Windows на Python 3.15.1 из существующей среды META. Это инженерные проверки новой реализации; они не измеряют универсальное качество модели или процент сходства с другим продуктом.

Файловые проверки охватывают недопустимые пути, защищённые имена, исключение крупных и скрытых файлов и отказ переходить через Windows junction. После 4.0.1 добавлена проверка создания вложенной папки только по разрешению, отказа выходить за Projects и ситуации, когда имя папки занято обычным файлом.

Проверка цикла использует заранее управляемые ответы модели, но запускает настоящие Python-тесты. Она проходит последовательность: чтение исходника, добавление теста, провал теста, исправление кода, успешная повторная проверка, создание многофайловой заявки, применение и восстановление прежнего содержимого. Это доказывает связность механизма обратной связи независимо от случайного поведения модели.

Отдельно проверены лимит шагов, отмена, требование чтения перед изменением, отказ при устаревших хэшах и восстановление после искусственной ошибки записи второго файла. При поздней ручной правке откат отказывается её затирать. Для запуска тестов проверены явное согласие, запрет неизвестного профиля, ноль найденных тестов, тайм-аут и ограничение вывода.

Проверка дерева процессов создаёт дочерний процесс с отложенной записью. После тайм-аута эта запись не происходит. Дополнительно проверены токен и параметры API, история после перезапуска, запрет второй задачи, отмена текущей, защитные проверки Desktop, установка и восстановление при сбое. Отдельный тест подтверждает, что исправление 4.0.1 сохраняет остальной JavaScript, повторно ничего не меняет и допускает точный откат.

Настоящая локальная модель

В отдельном демонстрационном запуске участвовала реальная qwen2.5-coder:14b через локальную Ollama. В calc.py функция add ошибочно вычитала числа; test_calc.py ожидал, что add(2, 3) вернёт 5. Модель последовательно прочитала оба файла, предложила замену вычитания сложением, запустила тест и завершила задачу. Потребовалось пять шагов; один unittest прошёл успешно.

В этом запуске модель исправила очевидную ошибку до первого тестирования. Поэтому утверждать, что именно настоящая модель прошла в демонстрации цепочку «провал → исправление → повторный успех», было бы неточно. Этот сценарий проверен отдельным автоматическим тестом. Демонстрация возвращала предварительный результат, а не применяла настоящую заявку в рабочем проекте.

Что ещё требует проверки пользователем

Синтаксис новой панели и установленного JavaScript проверен. Полный визуальный запуск обновлённого Qt-окна, применение заявки кнопкой в этом окне и сложные задачи на реальном пользовательском репозитории в данном чате не проверялись. Установочный процесс тестировался на временных образцах; проверка совместимости преобразований прошла на найденных реальных исходниках.

6 Установка и возможности отката

Как обновление встраивается в META

Основной установщик добавляет пакет meta40 и конфигурацию Core, подключает API в main.py и расширяет функции предварительного просмотра и подтверждения agent_pending.py. Desktop получает модуль meta40_desktop.py, обработчики обращений к новому Core API и код панели, добавляемый к assets/app.js. В первой сборке предусмотрено 12 добавляемых или обновляемых файлов.

Старый маршрут чата не заменяется агентным режимом. Для новых задач нужно открыть отдельную панель. Прежние компоненты 3.x остаются на месте и продолжают обрабатывать свои заявки. При такой интеграции можно развивать новый движок без полной переписи уже работающего приложения.

Что делает установщик перед записью

Он проверяет наличие ожидаемых файлов и точек подключения, разбирает подготавливаемый Python-код и сверяет границу Projects с WORKSPACE существующего agent_tools.py. Режим --check выполняет подготовку и проверку без записи. При неизвестной структуре исходников установка останавливается. Повторная полная установка поверх обнаруженного подключения 4.0 блокируется.

До замены сохраняются исходные байты изменяемых файлов и манифест. Для каждого файла фиксируются контрольные суммы до и после обновления. Если во время установки файл изменился, операция прекращается; если ошибка произошла после части замен, установщик пытается восстановить уже изменённые файлы.

Откат программного обновления

Откат запускается по точному manifest.json, который печатает установщик. Он проверяет границы целевых путей, целостность резервных копий и соответствие текущего файла установленной либо уже восстановленной версии. При более поздних ручных изменениях он останавливается, чтобы не перезаписать их без разбора. Добавленные программные файлы удаляются, прежние восстанавливаются.

Модели Ollama, история чатов и базы данных не входят в набор заменяемых файлов. Резервные копии и история нового агента остаются после отката. Программный откат не отменяет отдельно применённые правки пользовательских проектов.

Откат изменений конкретного проекта

После применения заявки сохраняется квитанция meta40_receipts с прежним содержимым файлов. Команда python -m meta40.proposals rollback ID_ЗАЯВКИ восстанавливает старые файлы и удаляет созданные этой заявкой. Более поздняя правка блокирует восстановление. Пустые каталоги могут остаться.

Замена нескольких файлов не является единой атомарной транзакцией. При ошибке записи предпринимается обратное восстановление; при аварийном выключении состояние разбирается по квитанции, в том числе со статусом applying. Механизм уменьшает риск потери работы, но не заменяет обычные резервные копии и контроль версий.

7 Исправление ошибки выбора проекта

Почему появилась ошибка

После установки пользователь сообщил сообщение «Project directory must already exist». Оно возникало до старта агентной задачи: код проверял выбранную папку, а папки ещё не было. В исходном режиме 4.0 поле проекта выбирало существующий каталог и не создавало его. Это было ограничением сценария начала работы, а не сообщением о сбое Ollama.

Например, имя my_app означало D:\META\Projects\my_app. Если пользователь ещё не создавал этот каталог, запуск прекращался. Значение «.» выбирало всю папку Projects. В интерфейсе первой версии не было достаточно удобного способа начать новый проект из этого поля.

Что изменено в 4.0.1

В панель добавлен пункт «Создать папку проекта, если её нет», включённый по умолчанию. При запуске интерфейс передаёт выбор пользователя в новый параметр create_project. Сам API по умолчанию оставляет создание выключенным: старые и внешние обращения не получают это поведение без отдельного флага.

Создание выполняется только внутри уже существующей корневой папки Projects. Имя проекта полностью проверяется заранее; вложенные каталоги создаются последовательно. На каждом существующем участке проверяется отсутствие ссылок и junction. Если имя занято файлом или путь запрещён, задача не запускается. Проверка занятости агента перенесена перед созданием каталога, чтобы отклонённая вторая задача не создавала лишние папки.

Сообщение об отсутствии каталога теперь содержит понятный русский текст и подсказку включить создание папки. Это улучшение относится к отсутствующей подпапке проекта. Саму настроенную корневую папку Projects нужно иметь заранее; неверный путь конфигурации исправление молча не создаёт.

Что реально установлено

Исправление применено к найденной установленной META отдельным patch_meta401.py. Заменены четыре файла: meta40/__init__.py, meta40/workspace.py, meta40/api.py и Desktop assets/app.js. Для новых маршрутов номер версии теперь выдаётся из VERSION со значением 4.0.1. Отдельные старые поля версии интерфейса или конфигурации могут по-прежнему отображать прежнюю маркировку.

Резервная копия исправления сохранена в D:\META\Core\meta40_backups\fix401_20261102_053457_781869. Повторная проверка установленных файлов показала ноль необходимых изменений; JavaScript установленного Desktop прошёл проверку синтаксиса. ZIP-пакет и инструкция также обновлены.

Как использовать после перезапуска

Нужно перезапустить и Core, и Desktop, открыть «Агент 4.0», ввести имя проекта, например my_app, оставить включённым создание папки и отправить задачу. Созданный пустой каталог не означает разрешения записывать сгенерированный код: для файлов остаётся отдельное подтверждение заявки. Тесты также требуют собственного согласия.

8 Чего мы добились и что остаётся

Практический результат для пользователя

Теперь META может выполнять небольшой самостоятельный цикл работы с проектом: получить задачу, изучить нужные исходники, найти место ошибки, подготовить изменения, проверить их Python-тестом и предложить человеку общий результат. Пользователю не нужно вручную пересылать каждый вывод инструмента модели. При этом перед применением остаётся точка контроля с реальным diff.

Вместо одного текстового ответа появился наблюдаемый процесс. Журнал показывает выбранные действия и тестовый вывод, историю можно открыть повторно, задачу можно отменить. Установка и применение файлов сопровождаются данными для восстановления. Ошибка создания нового проекта устранена отдельным установленным исправлением.

Что 4.0 пока не умеет

В новой реализации нет инструментов Git и GitHub, произвольного терминала, запуска npm или pytest, автоматической установки зависимостей, браузера документации, языкового сервера LSP, удаления и переименования файлов. Несколько агентов параллельно не работают: выполняется одна задача. Поддержка TypeScript как текстового файла не означает наличие компилятора, проверки типов или JavaScript-тестов.

Снимок намеренно ограничен. Он исключает .git, виртуальные окружения, node_modules, скрытые и двоичные файлы и тексты больше 20 000 байт. Поэтому большие исходники, сложные сборки и тесты с двоичными ресурсами могут быть недоступны. Python-тесты выполняются существующим интерпретатором META; новый движок не подбирает автоматически отдельное окружение каждого проекта.

Сохраняемая память представляет последнее резюме, а не поиск по долговременной базе знаний. Формат JSON и правила инструментов помогают управлять моделью, но не гарантируют её рассуждения. Она может выбрать неудачный шаг, неправильно понять задачу или исчерпать бюджет. Фактический вывод тестов важнее уверенного текстового резюме.

Почему нельзя заявить достижение 70 процентов

В этом чате не было формального набора одинаковых задач, прогона на большой выборке и сопоставления результатов с Claude Code. Поэтому число 70 процентов остаётся пожеланием к направлению развития. Наличие похожих инструментов не означает одинаковое качество программирования, работу с большим репозиторием или надёжность на длинных задачах. Возможности локальной Qwen и качество программного окружения следует оценивать отдельно.

Как оценивать следующий этап

Разумная следующая проверка — несколько реальных задач с заранее определённым результатом: исправить ошибку с тестом, изменить два связанных модуля, добавить небольшую функцию и проверить сохранение существующего поведения. Для каждой нужно фиксировать число шагов, результат тестов, необходимость помощи человека и корректность конечного diff. Это предлагаемый способ оценки, а не уже проведённый эксперимент.

После такой оценки можно выбирать дальнейшие улучшения: поддержку Git, настройку окружений проектов, дополнительные профили тестов, более богатый контекст и усиленную изоляцию исполнения. Эти пункты в этом чате не реализованы и не входят в текущие обещания сборки.

9 Состав результата и основные термины

Где находится подготовленная сборка

Рабочая папка результата: C:\Users\user\Documents\ChatGPT\META\META_4_0. Архив для передачи: META_4_0_Upgrade.zip в родительской папке META. Рядом с архивом хранится его SHA-256; внутри сборки есть SHA256SUMS.txt. Эти суммы позволяют обнаружить изменение файлов после упаковки.

Файл или группа	Назначение
core/meta40/engine.py	Цикл решений модели и обращения к Ollama
core/meta40/workspace.py	Пути, снимок, разрешённые файлы и создание проекта
core/meta40/runner.py	Управляемый запуск Python unittest
core/meta40/proposals.py	Заявки, проверка хэшей, применение и откат проекта
core/meta40/api.py	Состояния задач, фоновые потоки и маршруты Core
desktop/agent40.js и meta40_desktop.py	Панель и серверная связь Desktop с Core
install_meta40.py и Install_META_4.cmd	Первая установка, проверка и программный откат
patch_meta401.py	Установка исправления поверх существующей 4.0
tests/test_meta40.py	17 автоматических проверок
demo_ollama.py и LIVE_DEMO_RESULT.json	Живой пример Ollama и сохранённый результат
README_RU.md и VERIFICATION_RU.md	Инструкция, ограничения и сведения о проверках

Короткий словарь

Агентный цикл — повторение действий модели с получением результата каждого инструмента. Ollama — локальный сервис запуска модели. Снимок — набор исходных текстов, зафиксированный перед задачей. Слой правок — новые версии файлов, которые пока не записаны в рабочий проект.

Заявка — сохранённый набор предлагаемых изменений для подтверждения. Diff — отображение отличий между прежним и новым текстом. Хэш — контрольная сумма содержимого, позволяющая заметить изменение файла. Unittest — стандартный механизм тестирования Python. Job Object — средство Windows для управления группой процессов. Квитанция восстановления — журнал прежних байтов и сведений о применённой заявке.

Основание технических выводов

Отчёт опирается на сообщения и результаты команд этого чата, прочитанные исходники прежних agent_pending.py, agent_project37.py и meta_engine38.py, созданный пакет meta40, тесты, отчёт LIVE_DEMO_RESULT.json и квитанцию установки исправления. Секретный токен Core, пароли и содержимое пользовательских баз в документ не включены.